

Phát triển ứng dụng web

ReactJS
(part 4)

Nội dung

- ☐ Single Page Application
- ☐ React Router
- ☐ Route Objects & Data Router

Nội dung

- ☐ **Single Page Application**
- ☐ React Router
- ☐ Route Objects & Data Router

MPA - Multi Page Application

- ☐ Là kiến trúc Web truyền thống.
- ☐ Cơ chế:
 - ☐ Client gửi request (URL) -> Server xử lý -> Server trả về file HTML hoàn chỉnh.
 - ☐ Mỗi lần chuyển trang là một lần trình duyệt reload (tải lại) toàn bộ resources (CSS, JS).
- ☐ **Ưu điểm:** SEO tốt tự nhiên, lần tải đầu tiên nhanh.
- ☐ **Nhược điểm:** Trải nghiệm người dùng bị gián đoạn (trang trắng khi chuyển), server chịu tải lớn mỗi lần render.

Kiến trúc SPA (Single Page Application) & CSR

☐ Cơ chế:

- ☐ Chỉ có 01 file HTML duy nhất (**index.html**).
- ☐ Server trả về file JSON data và file JavaScript (Bundle).
- ☐ **Client-Side Rendering (CSR)**: Browser chạy JS để tự vẽ giao diện (DOM).

☐ **Đặc điểm**: Khi chuyển trang, thực chất là xóa Component cũ và mount Component mới. Không reload trang.

Vấn đề của "Routing" trong SPA

- ☐ Vấn đề: Vì chỉ có 1 file HTML, mặc định trình duyệt luôn hiển thị 1 URL duy nhất (ví dụ: **domain.com**).
- ☐ Do đó bị hạn chế:
 - ☐ Không thể chia sẻ link trang con.
 - ☐ Nút Back/Forward của trình duyệt bị vô hiệu hóa.
 - ☐ Người dùng F5 (Refresh) sẽ bị mất trạng thái hiện tại.
- ☐ **Giải pháp**: Cần một hệ thống giả lập việc điều hướng.

Routing trong SPA

- ❑ Mapping **URL** → **Component**
- ❑ Thay đổi **URL** nhưng **không reload** (history API)
- ❑ Lợi ích:
 - ❑ UX mượt (không nháy trang).
 - ❑ Dễ chia module/màn hình.
 - ❑ Tương thích back/forward của trình duyệt.

HTML5 History API - "Trái tim" của Router

- ❑ Router không phải ma thuật, nó dựa trên **window.history** của trình duyệt.
- ❑ **pushState(state, title, url)**: Thay đổi URL trên thanh địa chỉ nhưng **không reload trang**.
- ❑ **popstate event**: Sự kiện lắng nghe khi người dùng bấm Back/Forward.

Nội dung

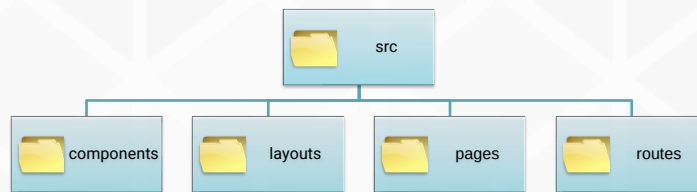
- ☐ *Single Page Application*
- ☐ **React Router**
- ☐ Route Objects & Data Router

React Router – Những khái niệm chính

- ☐ **Route:** “Luật” match URL → component
- ☐ **Router:** “Bộ điều phối”, lắng nghe URL đổi để render đúng route.
- ☐ **Link:** Thay thế **<a>** truyền thống, nhưng không reload
- ☐ **Hooks:**
 - ☐ `useNavigate`, `useParams`, `useLocation`, `useSearchParams`, ...
- ☐ **Nested Routes:**
 - ☐ Cho phép lồng layout & trang con

Tích hợp React Router

- ❑ Thư viện: **React Router DOM**
- ❑ Cài đặt: **npm i react-router-dom**
- ❑ Cấu trúc thư mục gợi ý



BrowserRouter vs HashRouter

- ❑ **BrowserRouter:**
 - ❑ URL đẹp: example.com/about
 - ❑ Yêu cầu cấu hình Server (Nginx/Apache) để **redirect** tất cả **request** về [index.html](http://example.com/index.html).
- ❑ **HashRouter:**
 - ❑ URL có dấu thăng: example.com/#/about
 - ❑ Dùng cho các host tĩnh cũ (Github Pages đời cũ) không hỗ trợ **redirect**.

Thiết lập context

- ❑ Bọc ứng dụng bằng **<BrowserRouter>**.

```
createRoot(document.getElementById("root")).render(  
  <StrictMode>  
    <BrowserRouter>  
      <Auth>  
        <App />  
      </Auth>  
    </BrowserRouter>  
  </StrictMode>  
>);
```

Định nghĩa Routes và Route

- ❑ **<Routes>**: Thay thế cho **Switch** ở v5. Nó chọn **route** khớp nhất dựa trên thuật toán tính điểm (ranking), không quan trọng thứ tự sắp xếp.
- ❑ **<Route>**:
 - ❑ **path**: Đường dẫn URL.
 - ❑ **element**: Component sẽ render.
 - ❑ **caseSensitive**: Phân biệt hoa thường (mặc định là false).

```
<Routes>  
  <Route path="/" element={<HomePage />} />  
  <Route path="/about" element={<AboutPage />} />  
</Routes>
```

Link và NavLink

❑ Component `<Link to="...">`:

- ❑ Render ra thẻ `<a>` trong DOM nhưng chặn sự kiện click mặc định.
- ❑ Gọi `history.pushState()`.
- ❑ Giữ nguyên **State** của ứng dụng (Redux/Context không bị reset).

❑ NavLink - Active Styling

- ❑ Dùng cho Menu/Navbar để highlight link hiện tại.
- ❑ **Tính năng:** Tự động biết mình có đang "active" hay không dựa trên URL hiện tại.

Ví dụ

```
<nav>
  <Link to="/">Home</Link>
  { " | " }
  <Link to="/about">About</Link>
</nav>
```

```
<nav className="flex gap-4">
  <NavLink
    to="/"
    className={({ isActive }) =>
      isActive ? "text-blue-600 font-bold" : "text-gray-600">
    Home
  </NavLink>
  <NavLink
    to="/about"
    className={({ isActive }) =>
      isActive ? "text-blue-600 font-bold" : "text-gray-600">
    About
  </NavLink>
</nav>
```

Truyền dữ liệu qua State của Route

- ❑ Route cho phép gửi kèm object ngầm là state.

```
<Link
  to="/register"
  className="text-sm text-blue-600 hover:underline"
  state={{ from: "login" }}>
  Sign Up
</Link>
```

- ❑ `useLocation` Hook

- ❑ Trả về object chứa thông tin URL hiện tại.
- ❑ Các thuộc tính: **pathname**, **search**, **hash**, **state**.

```
const location = useLocation();
const from = location.state?.from || "/";
```

useNavigate Hook

- ❑ Hook **useNavigate()**:

- ❑ Trả về hàm **navigate(pathOrDelta, options?)**

- ❑ Dùng trong xử lý logic:

- ❑ Sau khi login thành công → chuyển trang

```
const navigate = useNavigate();
navigate("/home"); // Push history
navigate("/login", { replace: true }); // Replace history
navigate(-1); // Tương đương nút Back
```

- ❑ Component **<Navigate />**

- ❑ Dùng để **Redirect** ngay khi render.
- ❑ Thường dùng trong Protected Route.

Xử lý 404 - Not Found Route

- ❑ Định nghĩa một Route với **path="*"**.
- ❑ Dấu ***** có độ ưu tiên thấp nhất, chỉ khớp khi không có route nào khác khớp.

```
<Route
  path="*"
  element={
    <Navigate to={isAuthenticated ? "/products" :
"/login"} replace />
  }
/>
```

Dynamic Routing

- ❑ Vấn đề: Không thể hard-code **/product/1**, **/product/2...**
- ❑ Định nghĩa: route dynamic
 - ❑ **/products/:id**
 - ❑ **/users/:userId/orders/:orderId**
- ❑ Hook **useParams()**:
 - ❑ Trả về object chứa params.
 - ❑ Luôn trả về chuỗi (String).

```
const { userId } = useParams();
```

Query string

- ❑ Dạng: **?name=abc&age=20**.
- ❑ Khác biệt: Params là bắt buộc (thay đổi cấu trúc), Query là tùy chọn (filter, sort).
- ❑ Hook **useSearchParams**:
 - ❑ Hoạt động giống **useState** nhưng đồng bộ với URL.
 - ❑ Trả về: **[searchParams, setSearchParams]**.

```
const [searchParams, setSearchParams] = useSearchParams();  
const page = searchParams.get("page");  
setSearchParams({ page: "2" });
```

Vấn đề "Component Mounting" với Route

- ❑ Tình huống: Đang ở **/product/1**, bấm link sang **/product/2**.
- ❑ Hành vi: **React không unmount component ProductDetail**, nó chỉ **update props/params**.
- ❑ Lưu ý: Cần dùng **useEffect** lắng nghe **[productId]** thay đổi để **fetch** lại dữ liệu mới.

```
useEffect(() => {  
  // Fetch product details using productId  
}, [productId]);
```

Layout trong React Router

❑ Component `<Outlet />`:

- ❑ Được đặt ở Component cha.
- ❑ Là vị trí render route con.

❑ Index Route (`index` prop)

- ❑ Chỉ định "Route con mặc định".

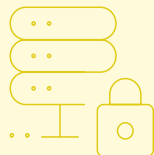
```
<>  
  <NavBar />  
  <main>  
    <Outlet />  
  </main>  
  <Footer />  
</>
```

```
<Route element={<MainLayout />}>  
  <Route index element={<Home />} />  
  <Route path="/products" element={<Products />} />  
</Route>
```

Protected Route (Auth Guard)

Ngăn chặn truy cập trái phép

Ngăn chặn người dùng chưa đăng nhập truy cập các trang nhạy cảm.



Không thể chỉ ẩn nút menu

Người dùng có thể bỏ qua các nút menu ẩn bằng cách nhập URL trực tiếp.



Cần cơ chế xác thực

Cần một cơ chế để kiểm tra xác thực và chuyển hướng người dùng chưa đăng nhập.



Kết hợp Outlet và Navigate

```
export const ProtectedRoute = ({ redirectTo =  
  "/login" }) => {  
  const { isAuthenticated } = useAuth();  
  if (!isAuthenticated) {  
    return <Navigate to={redirectTo} replace />;  
  }  
  return <Outlet />;  
};
```

```
<Route element={<ProtectedRoute />}>  
  <Route path="/profile" element={<ProfilePage />} />  
  <Route path="/settings" element={<SettingsPage />} />  
</Route>
```

Code Splitting & Lazy Loading

- ❑ Bundle size lớn làm chậm lần tải đầu (**FCP - First Contentful Paint**).
- ❑ **React.lazy()**: Biến import tĩnh (**import X from './X'**) thành import động.
- ❑ Suspense Component
 - ❑ **React.lazy** là bất đồng bộ.
 - ❑ **<Suspense fallback={<Spinner />}>** bao bọc Routes để hiển thị UI chờ trong lúc tải.

```
const HomePage = lazy(() => import("@/pages/Home.jsx"));
```

Nội dung

- ☐ *Single Page Application*
- ☐ *React Router*
- ☐ **Route Objects & Data Router**

UI Router

- ☐ Đặt routes trực tiếp trong JSX.
- ☐ Data fetching thường dùng: **useEffect + fetch** trong từng **component**.
- ☐ Vấn đề khi quy mô app lớn
 - ☐ Khó tách cấu hình routes ra file riêng.
 - ☐ Data: Mỗi component tự fetch → khó **prefetch, caching, error boundary** theo route.
 - ☐ Form: Tự handle onSubmit, preventDefault, fetch, redirect, lỗi...

Route Objects

❑ Cấu hình routes là **mảng các object**:

- ❑ **path**
- ❑ **element**
- ❑ **children**
- ❑ (Data Router): **loader, action, errorElement, id,...**

```
const routes = [  
  {  
    path: "/products",  
    element: <Products />,  
  },  
];
```

createBrowserRouter & RouterProvider

```
const router = createBrowserRouter([  
  {  
    path: "/",  
    element: <AppLayout />,  
    children: [  
      { index: true, element: <Home /> },  
      { path: "products", element: <Products /> },  
    ],  
  },  
]);
```

```
createRoot(document.getElementById("root")).render(  
  <StrictMode>  
    <RouterProvider router={router} />  
  </StrictMode>  
);
```

Data Router

❑ Loaders - Đọc dữ liệu

- ❑ Hàm **loader** chạy trước khi component render.
- ❑ Sử dụng hook **useLoaderData()** để lấy dữ liệu trong component.
- ❑ Giúp loại bỏ **useEffect** phức tạp.

❑ Actions - Ghi dữ liệu

- ❑ Hàm **action** xử lý Form Submission (POST/PUT/DELETE).
- ❑ Kết hợp với **<Form>** component của React Router.
- ❑ Tự động re-validate data sau khi action thành công.

Ví dụ

```
async function productsLoader({ request, params }) {  
  const res = await fetch(`${url}/rest/v1/products?select=*`, {  
    headers: {  
      apikey: apiKey,  
    },  
  });  
  const data = await res.json();  
  return data || [];  
}  
const router = createBrowserRouter([  
  {  
    path: "/",  
    element: <ProductsPage />,  
    loader: productsLoader,  
  },  
]);
```

```
const products = useLoaderData() || [];
```

ErrorElement - Xử lý lỗi cấp độ Route

- ❑ Mỗi Route object có prop **errorElement**.
- ❑ Nếu **loader** hoặc **component** bị lỗi, **errorElement** sẽ hiển thị thay thế.

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <AppLayout />,
    errorElement: <ErrorPage />, // cho toàn bộ subtree
    children: [
      {
        path: "products",
        element: <Products />,
        loader: productsLoader,
      },
    ],
  },
]);
```

JSX Route vs. Route Objects + Data Router

Tiêu chí	JSX Routes (Classic)	Route Objects + Data Router (Modern)
Cú pháp định nghĩa	Dùng Component JSX (<Route path="..." />)	Dùng Mảng Objects JavaScript ({ path: "..." })
API Khởi tạo	BrowserRouter	createBrowserRouter + RouterProvider
Chiến lược dữ liệu	Render-then-Fetch : Component hiện ra -> chạy useEffect -> mới gọi API.	Render-as-you-Fetch : Router gọi API (loader) song song lúc tải Component.
Vấn đề hiệu năng	Dễ gặp Waterfall (Component cha load xong con mới load).	Giải quyết Waterfall (Load song song).
Xử lý Form	Tự viết onSubmit, useState, gọi API thủ công trong component.	Tự động hóa qua action và <Form>. Tự động re-validate data.
Xử lý lỗi (Error)	Phải bọc thủ công ErrorBoundary bên ngoài component.	Tích hợp sẵn errorElement cho từng Route.
Loading State	Tự quản lý isLoading state (true/false).	Tự động qua navigation.state ("loading", "submitting").

Nội dung

- ☐ *Single Page Application*
- ☐ *React Router*
- ☐ *Route Objects & Data Router*
- ☐ **Bài tập**

Bài tập

- ☐ Với dữ liệu được cung cấp từ API, hãy chuyển bài tập tuần 07 với việc áp dụng React router.